

社内LT

公開されている **Agent Skills**を使ってみる

grill-meの紹介

Skills、自分で作り込んでいませんか？

モデルが変わったら、指示も見直す

以前のモデル向けに足した手順が、更新後も必要とは限らない。

作ったあとも、動作確認と修正が続く

指示を足すたびに、別のタスクでも使えるか確かめる。

自作する前に、公開**Skills**も探してみる。

公開されているSkillsも使える

GitHubなどで公開されている**Skill**を
Claude Codeなどに追加して使える。

手順を一から書かずに、公開されているものを試せる。

追加前に内容を確認し、手元のタスクで試す。

公開例 : Matt Pocock / mattpocock/skills (MITライセンス)

grill-me

質問に答えながら、曖昧な点や抜けている条件を確認する。

文書にも残したいときは **grill-with-docs**

質問に加えて、用語や設計判断も文書に残す。

出典

Matt Pocock / [mattpocock/skills](https://github.com/mattpocock/skills) (MITライセンス)

grill-me の追加と呼び出し

例 : **Claude Code** (コマンドは **Bash / zsh**)

1 作業フォルダーで追加する

```
npx skills@latest add mattpocock/skills \  
--skill grill-me grilling --agent claude-code
```

2 **CLAUDE CODE**のチャットで呼び出す

```
/grill-me
```

Azure Functionsで社内向けAPIを作りたいです。

方式設計Agentに渡す前に、認証と公開範囲を詰めたいです。

Node.js / npm が必要です。grilling は grill-me が内部で使う Skill です。

grill-me の会話例

質問に推奨案も付くので、回答を考えやすい。

AI

誰がAPIを呼び出しますか？

推奨案：まずは社内システム専用に絞る。

インターネットからのアクセスは必要ですか？

推奨案：社内ネットワークからの利用に限定する。

自分

社内バッチから呼びます。

社外からのアクセスは不要です。

説明用の会話例。実際の質問・推奨案・順序は変わります。

回答に合わせて、質問が続く

「社内バッチから呼ぶ」と分かったので、次を確認。

AI

バッチはどこで動いていますか？

推奨案：既存の実行環境をそのまま使う。

APIの認証方式に指定はありますか？

推奨案：既存のID基盤に合わせる。

自分

バッチはオンプレです。Azureへの閉域接続があります。

認証はEntra IDのアプリ認証に統一します。

説明用の会話例（前ページの続き）。

質問で詰めた要件を、方式設計Agentへ

回答をまとめて渡す

呼び出し元や接続・認証の条件を添えて、方式案の作成を依頼する。

未確認は未確認として残す

分からない条件は、未確認と明記する。

この例では未確認：**API**の処理内容、呼び出し量、可用性など。

デモをやってみます

| 今日の晩ご飯を **grill-me** で考える

質問が広がりすぎたとき

「今回は認証と公開範囲だけ決めたいです」

何を決めたいか、範囲を指定する。

「性能はまだ分かりません。検証項目にしてください」

分からないことまで、その場で決めなくてよい。

推奨案も確認は必要。製品仕様の調査や性能の実測は、別に行います。

参考資料

公開Skillsの一覧と追加方法

github.com/mattpocock/skills

質問に答えて、案や要件を詰める

[grill-me / SKILL.md](#)

用語や設計判断も文書に残す

[grill-with-docs / SKILL.md](#)

公開元 : Matt Pocock / [mattpocock/skills](https://github.com/mattpocock/skills) (MIT) 。公開内容は更新されることがあります。